Milestone 1 — Delivery Plan & Status

Milestone 1 ("the reliability spine") is split into three sub-milestones. The day ranges map to the build guide's 2-week sequence.

Sub-milestone	Days	Scope
M1.1 — Foundation & Data Layer	1–3	Schema + Prisma migrations, availability templates, materialized slots, state machine, DB-level constraints
M1.2 — Booking Core & Concurrency	4–7	Row-level locking (`SELECT … FOR UPDATE`), reservation expiry worker + sweep, core booking flow end-to-end, admin CRUD
M1.3 — Payments, Verification & Delivery	8–14	Pix + card integration, signature-verified idempotent webhooks, expiry/payment race handling, concurrency + webhook test suites, deployment (Railway/Render), docs + Postman collection, final hardening

The four client acceptance criteria (no double booking, reliable expiry release, end-to-end Pix + card in sandbox, reproducible setup) are proven in M1.3 but enabled by the constraints and state machine laid down in M1.1.

Environment note (verification on this machine)

This dev machine has no Node/Docker preinstalled and its `D:` drive blocks native-binary writes (Defender). Verification therefore runs from a `C:` mirror with a portable Node 22 and a portable PostgreSQL 16 (no admin). One hard limit: **Prisma's Rust query engine crashes on this CPU** (`0xC000001D` illegal instruction / `0xC0000409` stack overrun, on both `library` and `binary` engines). That means the Prisma-based server and the Prisma integration tests (`test:integration`) cannot run *here* — they run in CI / on Railway/Render, where the Linux engine is fine.

To prove the runtime guarantees on this box regardless, the two critical behaviours are demonstrated with the pure-JS `pg` driver running the **identical SQL** the app uses:

Proof	Command	Result
Concurrency (criterion #1)	`npm run verify:locking`	✅ 50 contenders × 20 rounds — every round exactly 1 success, 49 clean losses, 0 errors
Expiry release (criterion #3)	`npm run verify:expiry`	✅ expired→freed, not-due→untouched, paid→never released

---

M1.1 — Foundation & Data Layer — status: ✅ VERIFIED

Delivered in this sub-milestone:

- [x] Project scaffold (TypeScript, ESM, npm scripts, Docker Postgres, env template).
- [x] Prisma schema: `Doctor`, `AvailabilityTemplate`, `AppointmentSlot`, `Reservation`, `Payment`, `WebhookEvent`, `EventLog`, plus the three status enums.
- [x] `init` migration (all tables, indexes, FKs) authored to match the schema.
- [x] `hard_constraints` migration — the guarantees Prisma can't express:
  - `one_live_slot_hold` partial unique index (a slot holds ≤ 1 live claim).
  - `no_overlap_per_doctor` GiST exclusion constraint (needs `btree_gist`) — a doctor can never have two overlapping slots.
- [x] Config plumbing: zod-validated env loader, pino logger (with secret redaction), Prisma client singleton, transactional `logEvent` helper.
- [x] Reservation/slot **state machine** encoded as data (`canTransition` / `assertTransition`) + full unit-test coverage of legal & illegal moves.
- [x] Availability templates → **slot materialization**: pure slot math (`iterateSlots`) + idempotent DB upsert (`materializeSlots`) over a rolling `SLOT_MATERIALIZATION_WEEKS` window, with limited-Sunday-hours handling.
- [x] Unit tests for slot math (spacing, custom slot length, trailing-partial drop, narrow Sunday window, empty/inverted windows).
- [x] Idempotent seed (demo doctor + weekday/Saturday/Sunday templates + materialized slots).

Verified: unit tests (12/12), full `tsc --noEmit`, migrations applied to a real Postgres, and the two hard constraints **observed rejecting bad data** (overlapping slot → rejected by `no_overlap_per_doctor`; duplicate `(doctorId, startsAt)` → rejected by the unique index; non-overlapping → accepted).

M1.2 — Booking Core & Concurrency — status: ✅ code complete, core guarantees proven

Delivered:

- [x] `reserveSlot` — raw `SELECT … FOR UPDATE` inside a Prisma interactive transaction, with lock-wait tx options tuned for concurrent bursts.
- [x] **Concurrency proven**: 50 × 20 rounds, exactly one winner every round (`npm run verify:locking`).
- [x] Expiry: `releaseReservation` (locked re-check), periodic `sweep`, precise pg-boss `scheduleExpiry`, and the `worker.ts` entrypoint wiring all three plus nightly materialization. **Release logic proven** across all three cases (`npm run verify:expiry`).
- [x] Reservation service (`reserve` + best-effort schedule; sweep is backstop).

- [x] Patient booking API — `GET /api/doctors`, `GET /api/slots`, `POST /api/reservations`, `GET /api/reservations/:id`, payment-start stub (501 until M1.3), plus `/healthz` + `/readyz`.
- [x] Admin CRUD behind an API-key guard — doctors, availability templates, on-demand materialize.
- [x] Central error handling (409 for lost races, 400 for bad input, never 500), zod validation, async handler wrapper.
- [x] Prisma integration tests authored (`tests/integration/reserve-race`, `tests/integration/expiry`) for CI/deploy where the Prisma engine runs.
- [x] Whole M1.2 tree typechecks clean (`tsc --noEmit` exit 0).

Not runnable on this machine: the Express server and `test:integration` (both need the Prisma runtime — see the environment note above). They are typechecked here and run in CI / deployment.

M1.3 — Payments, Verification & Delivery — status: 🟡 payment layer complete + proven; finalization pending

Provider chosen: **AbacatePay** (Pix-first). Built provider-agnostic so only the concrete adapter changes when sandbox keys arrive.

Delivered + proven:

- [x] Swappable `PaymentProvider` interface + AbacatePay adapter skeleton (endpoints/fields/signature scheme marked `TODO(sandbox)`) + in-memory `MockProvider` + provider registry.
- [x] HMAC-SHA256 raw-body signature verification (constant-time compare).
- [x] `startPayment` — stable idempotency key per (reservation, method); one Payment row; provider-idempotent charge.
- [x] Signature-verified, idempotent **webhook handler** with the full exception matrix, all under the same row lock the expiry job uses.
- [x] Raw-body webhook route mounted before `express.json()`; real payment-start route; 401 on tampered signatures; 409 on unpayable reservations.
- [x] **Proven on this box** (`npm run verify:webhook`): idempotency (5× sequential + 5× concurrent → confirm once, charge once), out-of-order convergence, tampered-signature rejection, and the **payment-at-expiry race consistent across 50 rounds** (always Paid+Approved OR freed+Refunded — never paid-but-unbooked).
- [x] Prisma integration tests authored (`tests/integration/webhook`) for CI.
- [x] Whole tree + tests typecheck clean (`tsc --noEmit`, `tsc -p tsconfig.test.json`).

Finalization still to do (the PDF's close-out phase):

- [ ] Wire the AbacatePay adapter against the real sandbox (needs API keys) and run one live Pix + one live card end-to-end with captured evidence.

- [ ] Security hardening: `helmet`, `express-rate-limit` on reserve/payment, HTTPS-only in prod, DB least-privilege role.
- [ ] k6 load suite in CI + docs package (`SCHEMA.md`, `API.md`, `WEBHOOKS.md`, `DEPLOYMENT.md`) + Postman collection.
- [ ] Deploy web + worker to Railway/Render; reproducibility check on a clean machine; client acceptance walkthrough + written sign-off.